

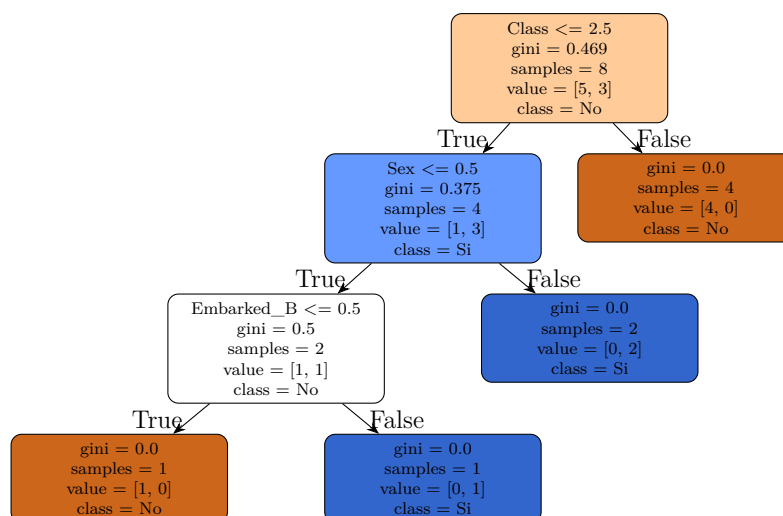
Guía de Ejercicios Random Forest

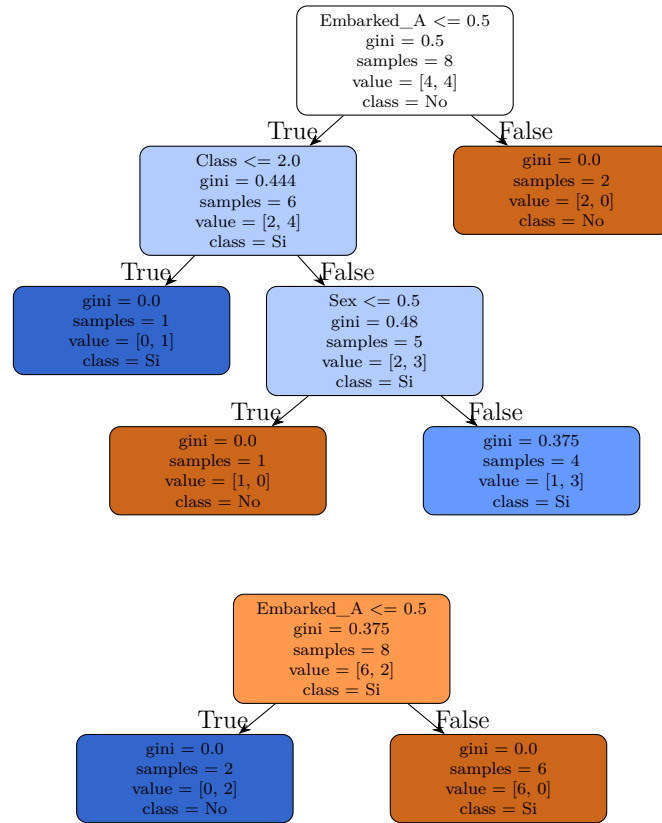
Analítica de Datos, Universidad de San Andrés

Si encuentran algún error en el documento o hay alguna duda, mandenme un mail a rodriguezr@udesa.edu.ar y lo revisamos.

Ejercicios

1. ¿Cómo deben ser los árboles que componen un modelo de random forest? ¿Por qué? ¿Cómo hacemos para conseguirlo?
2. Con este tipo de modelos ganamos poder predictivo pero perdemos interpretabilidad. ¿Qué podemos hacer en python para comprender la relevancia que tuvo cada variable al momento del entrenamiento?
3. Teniendo en cuenta el siguiente modelo de random forest clasificar a las siguientes observaciones indicando la probabilidad.
 - a) Sex = 0; Class = 1; Embarked = A
 - b) Sex = 1; Class = 2; Embarked = A





4. Entrenar un modelo de random forest para predecir si un animal es doméstico con el siguiente dataset de entrenamiento y teniendo en cuenta los siguientes hiperparámetros:

- Cantidad de árboles = 3
- Altura máxima de cada árbol = 2
- Cantidad de variables a tener en cuenta en cada partición = $\sqrt{p}$

ID	tiene_pelo	vuela	pone_huevos	vive_en_casa	domestico
1	sí	no	no	sí	sí
2	no	sí	sí	no	no
3	sí	no	sí	no	no
4	no	no	sí	no	no
5	sí	no	no	sí	sí
6	no	sí	sí	sí	sí
7	sí	no	no	no	no
8	no	no	sí	sí	sí
9	sí	no	sí	sí	sí
10	no	sí	sí	no	no

Muestras bootstrap para cada árbol:

- Árbol 1: [6, 9, 4, 1, 5, 3, 3, 1, 8, 1]
- Árbol 2: [4, 1, 8, 5, 1, 7, 7, 7, 1, 5]
- Árbol 3: [3, 7, 1, 6, 6, 6, 1, 7, 3, 6]

La elección aleatoria de variables la pueden hacer en python.

Soluciones

1. Los árboles deben ser diversos y débiles individualmente. Se logra mediante bootstrap sampling (muestras con reemplazo), selección aleatoria de variables ($\sqrt{p}$ en cada split) y sin poda para permitir overfitting individual. Al promediar múltiples predictores diversos, se reduce la varianza total manteniendo bajo el sesgo.
2. En Python usamos `rf.feature_importances_` (basado en reducción de impureza) o `permutation_importance` (más robusta, mide degradación del modelo al permutar cada variable):

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.inspection import permutation_importance
3
4 rf = RandomForestClassifier()
5 rf.fit(X_train, y_train)
6
7 # Metodo 1: Importancia por impureza
8 importancias = rf.feature_importances_
9
10 # Metodo 2: Permutation importance
11 perm_imp = permutation_importance(rf, X_test, y_test,
12                                   n_repeats=10)
```

3. Variables codificadas: Sex (0=mujer, 1=hombre), Embarked_A (1 si A, 0 si no), Embarked_B (1 si B, 0 si no).

Observación 1 (Sex=0, Class=1, Embarked=A):

- Árbol 1: $\text{Class} \leq 2.5 \rightarrow \text{Sex} \leq 0.5 \rightarrow \text{Embarked_B} \leq 0.5 \rightarrow \text{No}$
- Árbol 2: $\text{Embarked_A} \leq 0.5 \text{ (No)} \rightarrow \text{No}$
- Árbol 3: $\text{Embarked_A} \leq 0.5 \text{ (No)} \rightarrow \text{No}$

Predicción: No (probabilidad 1.0)

Observación 2 (Sex=1, Class=2, Embarked=A):

- Árbol 1: $\text{Class} \leq 2.5 \rightarrow \text{Sex} \leq 0.5 \text{ (No)} \rightarrow \text{Si}$
- Árbol 2: $\text{Embarked_A} \leq 0.5 \text{ (No)} \rightarrow \text{No}$
- Árbol 3: $\text{Embarked_A} \leq 0.5 \text{ (No)} \rightarrow \text{No}$

Predicción: No (prob. 0.67), Si (prob. 0.33)

4. Dataset convertido a binario (sí=1, no=0). Con $\sqrt{4} = 2$ variables por split y altura máxima 2, se construyen 3 árboles usando los bootstraps dados. La construcción manual requiere selección aleatoria de 2 variables en cada nodo. La idea es que hagamos nosotros el modelo de Random Forest sin usar el RandomForestClassifier de sklearn.

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.tree import DecisionTreeClassifier
4
5 # Crear dataset
6 data = {
7     'pelo': [1,0,1,0,1,0,1,0,1,0],
8     'vuela': [0,1,0,0,0,1,0,0,0,1],
9     'huevos': [0,1,1,1,0,1,0,1,1,1],
10    'casa': [1,0,0,0,1,1,0,1,1,0],
11    'domestico': [1,0,0,0,1,1,0,1,1,0]
12 }
13
14 df = pd.DataFrame(data)
15 X = df[['pelo', 'vuela', 'huevos', 'casa']]
16 y = df['domestico']
17
18 # Bootstraps dados
19 bootstrap1 = [5,8,3,0,4,2,2,0,7,0] # IDs - 1 (0-indexed)
20 bootstrap2 = [3,0,7,4,0,6,6,6,0,4]
21 bootstrap3 = [2,6,0,5,5,5,0,6,2,5]
22
23 # Entrenar arboles individuales
24 trees = []
25 for i, bootstrap in enumerate([bootstrap1, bootstrap2,
26     bootstrap3]):
27     X_boot = X.iloc[bootstrap]
28     y_boot = y.iloc[bootstrap]
29
30     tree = DecisionTreeClassifier(
31         max_depth=2,
32         max_features=2, # sqrt(4) = 2
33         random_state=i
34     )
35     tree.fit(X_boot, y_boot)
36     trees.append(tree)
```

```

37 # Hacer predicciones con el Random Forest
38 def predict_random_forest(X_new, trees):
39     predicciones = []
40     for tree in trees:
41         pred = tree.predict(X_new)[0] # Solo una
observacion
42         predicciones.append(pred)
43
44     # Votar por mayoria simple
45     votos_si = sum(predicciones)
46     votos_no = len(predicciones) - votos_si
47
48     if votos_si > votos_no:
49         return 1 # domestico
50     else:
51         return 0 # no domestico
52
53 # Ejemplo de prediccion
54 nueva_obs = [[1, 0, 0, 1]] # pelo=si, vuela=no, huevos=
no, casa=si
55 prediccion = predict_random_forest(nueva_obs, trees)
56 print(f"Prediccion: {'domestico' if prediccion == 1 else
'no domestico'}")
57

```